交付物 2：“架构辩论赛”记录与最终决策

1. AI 辅助架构调研记录

为了确保架构决策的客观性与科学性，团队 4 位成员分别采用了不同的 Prompt 策略，向 AI 专家（大语言模型）咨询了针对 CampusHub 项目的架构建议。以下为调研过程的核心记录：

1.1 成员 A：直接提问法 (Direct Inquiry)

Prompt 策略：不预设技术栈与团队背景，仅提供业务需求，直接询问推荐架构。

输入摘要：“我们正在开发 CampusHub 校园应用，提供信息汇总、活动发布和资源共享等服务，推荐什么架构？”

AI 反馈核心观点：

推荐架构：前后端分离的模块化单体架构 (Modular Monolith)。

技术栈建议：Vue 3 + Spring Boot + MySQL + Redis。

核心理由：微服务在初期会带来不必要的运维负担；模块化单体既能保持代码清晰边界，又能平滑过渡到微服务。建议尽早引入 Docker 容器化以解决环境不一致问题。

1.2 成员 B：约束条件注入法 (Constraint-based Inquiry)

Prompt 策略：详细输入团队规模（4人）、开发周期（10周）、技术栈（Vue3+SpringBoot）、缺乏分布式经验等硬性约束，要求 AI 进行针对性论证。

输入摘要：“基于 4人团队、10周工期、Vue3+SpringBoot、可维护性优先等约束，推荐最适合的架构并论证降低交付风险的策略。”

AI 反馈核心观点：

推荐架构：单体分层架构 (Monolithic Layered Architecture)，严格遵循 MVC 模式。

核心理由：学习成本极低，完美匹配大二学生的知识储备；规避了分布式复杂性，最大化开发效率。

降险策略：通过 Swagger 定义 API 契约实现前后端并行开发；单体 Docker 镜像部署降低环境风险；聚焦核心 PO 级功能（用户、需求大厅、订单）。

1.3 成员 C：多方案对比法 (Comparative Analysis)

Prompt 策略：要求 AI 对业界主流的多种架构（单体、模块化单体、微服务、Serverless、六边形架构）在特定约束下进行横向评测。

输入摘要："请基于项目约束，从学习成本、开发效率、交付风险等维度对比单体、模块化单体、微服务等 5 种架构，并给出推荐。"

AI 反馈核心观点：

推荐排序：首推模块化单体，次推单体分层；明确反对微服务与 Serverless。

核心理由：模块化单体兼顾了快速交付与长期可维护性。

落地建议：后端采用"按功能划分 (Package-by-feature)"的包结构；结合六边形架构原则，在模块内部保持解耦；坚持前后端契约优先。

1.4 成员 D：角色扮演法 (Role-playing / Persona)

Prompt 策略：设定 AI 为拥有 10 年经验的资深首席架构师，要求其以务实、落地的视角，对经典单体与微服务进行深度剖析。

输入摘要："你是一位资深架构师，请基于我们的学生团队背景和 10 周限期，深度对比单体与微服务，并给出最强烈推荐及务实理由。"

AI 反央核心观点：

推荐架构：强烈且唯一推荐「前后端分离的经典单体架构」，但必须辅以「模块化」代码规范。

核心理由："生存第一，炫技第二"。微服务大概率会导致项目无法按期交付。Vue 3 + Spring Boot 单体架构足以支撑校园级并发。

工程规范：严禁跨模块的数据库表直接 `JOIN` 查询，模块间必须通过 Interface 接口调用，为未来演进留出"口子"。

---

2. 团队辩论与最终架构决策

在汇总了 AI 的多维分析后，团队召开了架构辩论会。经过深入探讨，团队达成高度共识，做出如下最终决策：

🏆 最终决策：模块化单体架构 (Modular Monolith)

决策依据与论证：

交付确定性（10周红线）：
本项目最大的风险在于**时间逾期**。模块化单体架构完全避开了分布式事务、服务注册发现、链路追踪等复杂的"基建大坑"，使团队能将 90% 的精力聚焦于核心业务逻辑（用户、活动、订单）的实现，确保在 10 周内拿出可运行的 MVP 产品。

团队能力匹配度：
该架构完美契合大二学生的 Java/Vue 基础。代码联调在本地 IDEA 即可完成，无需配置复杂的 DevOps 环境，学习曲线平缓，开发效率最高。

可维护性与未来演进（核心质量要求）：
相比于传统的"面条式"单体代码，模块化单体通过严格的"按功能分包"和接口隔离，实现了逻辑上的高度解耦。这不仅便于期末交付后的代码交接，也为未来项目规模扩大时"平滑拆分为微服务"奠定了坚实的工程基础。

---

3. 技术栈选型与落地执行建议

为确保架构决策的顺利落地，团队制定了以下技术路线与工程规范：

3.1 核心技术栈

前端展现层： Vue 3 (纯 JavaScript) + Element Plus + Vue Router + Pinia。

后端业务层： Spring Boot 3.x (Java 17) + Maven + MyBatis-Plus。

数据存储层： MySQL 8.x（主业务数据）。

性能与缓存： Redis（用于热点数据缓存、Session 存储及防超卖流量削峰）。

部署与运维： Docker + Docker Compose（实现单节点一键容器化部署）。

3.2 模块划分与隔离规范

系统后端将划分为四大核心领域模块：`User`（用户中心）、`Event`（活动发布）、`Resource`（资源共享）和 `Notice`（实时通知）。

隔离红线： 严禁跨模块直接调用 Repository（数据库层）或进行跨业务表的直接 `JOIN` 查询。

通信规范： 模块间的数据交互必须通过暴露的 Service Interface（如 `Event` 模块需获取用户信息，必须调用 `UserService` 接口）。

3.3 研发流程规范

契约先行 (API First)： 在正式编写后端业务代码前，必须先定义并评审 RESTful API 文档（使用 Swagger/Knife4j）。前端基于 Mock 数据并行开发 UI。

CI/CD 自动化： 尽早配置简单的 GitHub Actions 脚本，实现代码推送到主分支后的自动构建与 Docker 镜